Séance 1 — Cloud Azure & conteneurisation

Master 2 · YNOV Campus Montpellier · Industrialisation de l'IA dans le Cloud · 2026-2027 Plateforme : Microsoft Azure · Énoncé apprenant

---

En-tête de séance

Élément	Détail
Durée	3 h 30 de TP
Modalité	Individuel, poste personnel, compte Azure for Students
Marche ajoutée	Modèles de service cloud, Docker, Azure Container Registry
Hors périmètre	Aucun modèle de machine learning, aucune API d'inférence (séance 2), aucun service managé déployé (séance 3)
Livrable	Un dossier `s1-<votre-nom>/` (voir section « Livrable attendu »)

Prérequis

Vérifiez ces quatre points **avant** le début de la séance. Un prérequis manquant coûte 20 minutes de TP.

1. Un compte **Azure for Students** actif (crédit de 100 $, sans carte bancaire), **créé avant la séance** : voir `Guide_Compte_Azure_Students.pdf`, à côté de cet énoncé. Vous devez arriver avec les quatre lignes de son tableau final renseignées. Attention au nom de votre abonnement : « Kit de démarrage Azure pour les étudiants » est l'offre *Starter*, sans crédit, sur laquelle le module ne tourne pas.
2. **Azure CLI** installé (`az version` doit répondre). Version 2.60 ou supérieure.
3. **Docker Desktop** installé et **démarré** (`docker info` doit répondre sans erreur).
4. Un éditeur de texte et un terminal. Sous Windows, utilisez PowerShell ou WSL2 — pas l'invite de commandes historique.

Objectifs

À la fin de la séance, vous savez :

- situer un service Azure dans la grille IaaS / PaaS / SaaS et nommer ce que vous n'avez plus à gérer ;
- relier les cinq caractéristiques NIST du cloud à des éléments observables dans le portail Azure ;
- écrire un Dockerfile correct pour une application Python, et justifier l'ordre de ses instructions ;
- mesurer l'effet du cache de build et la taille d'une image ;
- publier une image sur un registre privé Azure Container Registry et l'y retrouver ;
- documenter un conteneur dans une fiche d'identité exploitable par un tiers ;
- décider explicitement de ce que vous détruisez et de ce que vous conservez, et tracer cette décision pour ne pas consommer votre crédit à votre insu.

Contexte métier — le fil rouge du module

Vous venez d'être recruté comme Data/ML Engineer chez **DermaScan**, une société qui outille des centres de dépistage du cancer de la peau. Ses praticiens photographient des lésions cutanées et en relèvent des mesures cliniques ; l'objectif de l'entreprise est de **hiérarchiser** ces lésions — orienter vers un examen approfondi celles qui le méritent, laisser les autres en surveillance — pour que le temps de dermatologue aille là où il compte.

Le problème est rare et déséquilibré : dans les données réelles sur lesquelles vous travaillerez dès la séance 2, **une lésion sur mille vingt est maligne**. Ce chiffre n'est pas une difficulté de plus, c'est le sujet du module.

L'équipe data science a déjà des modèles qui fonctionnent sur les postes des data scientists. Aucun ne tourne en production. Votre mission sur le semestre est de construire la chaîne qui les y amènera.

On ne commence pas par le modèle. On commence par le **socle** : savoir emballer n'importe quelle application dans un artefact reproductible, et savoir où l'entreposer. Aujourd'hui, vous emballez le composant le plus simple de la future plateforme — une **sonde de santé** (`health probe`), un service HTTP qui répond un JSON indiquant qu'il est vivant. Aucun modèle dedans, et c'est volontaire : vous apprenez la conteneurisation sans la mélanger à l'entraînement d'un modèle.

En séance 2, cette même image servira de base à votre API d'inférence. Ce que vous produisez aujourd'hui n'est pas un exercice jetable.

Convention de lecture de l'énoncé

Deux notations, à ne pas confondre :

Notation	Signification	Quand la remplir
`________`	**Valeur à décider par vous** avant d'exécuter la commande	Avant l'exécution
Cellule vide d'un tableau de mesures	**Valeur à relever** dans la sortie d'une commande	Après l'exécution

L'énoncé n'affirme aucun chiffre : les tailles d'image, les durées de build et les nombres de couches sont **vos** mesures, sur **votre** machine. Deux apprenants n'obtiendront pas exactement les mêmes valeurs, et c'est normal.

Chaque commande est donnée avec un commentaire qui explique ce qu'elle fait. Lisez le commentaire avant de taper la commande.

Tableau de décisions

Complétez ce tableau **maintenant**, avant la partie 1. Toutes les commandes du TP s'y réfèrent. Vos choix vous engagent pour toute la séance : si vous changez d'avis en cours de route, vous casserez vos propres commandes.

Contraintes de nommage Azure à respecter :

- un nom de groupe de ressources accepte lettres, chiffres, `-` et `_` ;

- un nom de registre ACR est **globalement unique sur Azure**, en **minuscules et chiffres uniquement**, sans tiret, 5 à 50 caractères ;
- votre suffixe personnel sert à éviter les collisions avec vos camarades : prenez vos initiales suivies de deux chiffres, par exemple `ab42`.

Décision	Contrainte	Votre valeur
Suffixe personnel	4 à 6 caractères, minuscules et chiffres	`_____`
Région Azure	une seule, la même partout	`_____`
Nom du groupe de ressources	préfixé `rg-dermascan-registre-`	`_____`
Nom du registre ACR	minuscules et chiffres, unique sur Azure	`_____`
Nom de l'image	ce que l'image contient, pas votre nom	`_____`
Tag de version initial	versionnage sémantique	`_____`
Image de base Python	variante légère, version explicitement fixée, jamais `latest`	`_____`
Répertoire de travail dans l'image	chemin absolu, le même dans le `Dockerfile` et le `CMD`	`_____`
Port d'écoute de l'application	cohérent entre `EXPOSE`, la variable `PORT` et vos `-p`	`_____`
uid de l'utilisateur d'exécution	non nul, au-delà de 10000	`_____`

Question 0.1 — Le nom du registre ACR doit être unique sur l'ensemble d'Azure, pas seulement dans votre abonnement. Quelle contrainte technique ce choix trahit-il sur la manière dont Azure expose les registres ? Répondez en une phrase dans `reponses_partie1.md`.

Partie 1 — Fondamentaux du cloud

Cette partie se fait **en autonomie, tableau par tableau**. Ce n'est pas un cours magistral : vous cherchez, vous décidez, vous écrivez. Le formateur passe et arbitre les désaccords.

Créez un fichier `reponses_partie1.md` dans votre dossier de rendu et reportez-y les tableaux complétés.

1.1 — Les trois modèles de service

La première ligne est remplie à titre d'exemple. Complétez les deux autres.

Modèle	Ce que le fournisseur gère à votre place	Ce qui reste à votre charge	Exemple de service Azure
IaaS	Le matériel, le réseau physique, la couche de virtualisation	Le système d'exploitation, les mises à jour, le runtime, l'application, les données	Machine virtuelle (Azure Virtual Machines)
PaaS	_____	_____	_____
SaaS	_____	_____	_____

1.2 — Où s'arrête votre responsabilité

Pour chacune des couches suivantes, indiquez le **modèle le plus abstrait** (IaaS, PaaS ou SaaS) dans lequel cette couche reste encore de **votre** responsabilité. Exemple de lecture : si une couche est à votre charge en IaaS mais plus en PaaS, la réponse est « IaaS ».

Couche	Dernier modèle où c'est à vous
Correctifs de sécurité du système d'exploitation	_____
Version du runtime Python	_____
Code de l'application	_____
Dimensionnement du nombre d'instances	_____
Vos données métier	_____

Question 1.1 — Un seul élément de ce tableau reste à votre charge dans les trois modèles. Lequel, et quelle conséquence cela a-t-il sur la conformité RGPD d'un projet d'IA ? Trois lignes maximum.

1.3 — Les cinq caractéristiques NIST

Le NIST (SP 800-145) définit le cloud par cinq caractéristiques essentielles. La première ligne est remplie à titre d'exemple : traduisez les quatre autres en une phrase opérationnelle, dans vos mots, en évitant de recopier une définition trouvée en ligne.

Caractéristique NIST	Traduction opérationnelle en une phrase
Libre-service à la demande	Je crée une ressource sans ouvrir de ticket ni attendre l'accord d'un humain.
Accès réseau large	_____

Mise en commun des ressources	`_____`
Élasticité rapide	`_____`
Service mesuré	`_____`

Question 1.2 — Parmi ces cinq caractéristiques, laquelle explique directement que le nettoyage en fin de TP soit obligatoire ? Justifiez en deux lignes.

1.4 — Exploration du portail Azure

Connectez-vous sur portal.azure.com avec votre compte étudiant.

Mesure 1 — relevé d'exploration du portail Azure : abonnement, groupes de ressources, crédit restant et quotas. Effectuez ces six observations et notez vos réponses :

1. Dans la barre de recherche, tapez « Container Registries ». **Ne créez rien.** Ouvrez l'écran de création et relevez les champs obligatoires demandés. Combien y en a-t-il ?
2. Dans le menu latéral, ouvrez **Abonnements**. Quel est l'identifiant de votre abonnement et quel est son état ?
3. Ouvrez **Groupes de ressources**. Combien en existe-t-il actuellement dans votre abonnement ?
4. Ouvrez **Cost Management + Billing** puis l'analyse des coûts. Quel est votre crédit restant, à la date d'aujourd'hui ?
5. Ouvrez **Aide + support** puis les quotas de l'abonnement. Relevez un quota qui vous limiterait si vous vouliez créer dix machines virtuelles.
6. Dans une ressource quelconque, repérez l'onglet **Étiquettes** (`Tags`). À quoi sert-il, en une phrase ?

Classement. Pour chaque service, indiquez le modèle de service auquel il appartient. La première ligne est remplie.

Service Azure	Modèle
Azure Virtual Machines	IaaS
Azure App Service	`_____`
Azure Container Registry	`_____`
Azure Machine Learning (espace de travail managé)	`_____`
Microsoft 365	`_____`

Le groupe de ressources. Un groupe de ressources (`resource group`) est un conteneur logique qui regroupe des ressources partageant le même cycle de vie.

Question 1.3 — Répondez à ces trois questions dans `reponses_partie1.md` :

- Une ressource peut-elle appartenir à deux groupes de ressources simultanément ?
- Que se passe-t-il pour les ressources contenues lorsqu'on supprime un groupe de ressources ?
- La région d'un groupe de ressources contraint-elle la région des ressources qu'il contient ? Si non, à quoi sert-elle ?

Ces trois réponses conditionnent la partie 2 et le nettoyage final. Ne les survolez pas.

Partie 2 — Mise en place de l'environnement

Objectif : un terminal authentifié sur le bon abonnement, un groupe de ressources créé, un Docker fonctionnel. Rien de plus.

2.1 — Authentification Azure CLI

```
# Version du CLI. Si la commande échoue, le CLI n'est pas installé ou pas dans le PATH.
az version
# Authentification interactive : ouvre une fenêtre de navigateur.
az login
# Repérez la colonne IsDefault : l'abonnement sur lequel vos commandes s'appliqueront.
az account list --output table
# Fixez l'abonnement explicitement : sur un compte multi-abonnements, l'erreur est
# silencieuse et vous créez des ressources facturées ailleurs.
az account set --subscription "_______"
# Contrôle : abonnement réellement actif et identité connectée.
az account show --output table
```

Mesure 2 — identité du compte Azure. À relever : l'identifiant de votre abonnement et le nom du locataire (`tenant`). Vous en aurez besoin en partie 4.

2.2 — Choix de la région et groupe de ressources

```
# Régions ouvertes à votre abonnement : un compte étudiant n'en voit qu'un sous-ensemble.
az account list-locations --output table
```

Question 2.1 — Combien de régions votre abonnement expose-t-il ? Notez le chiffre. Citez deux critères qui, en entreprise, imposent le choix d'une région plutôt qu'une autre pour une plateforme traitant des données industrielles européennes.

Déclarez maintenant vos décisions sous forme de variables d'environnement. Vous éviterez ainsi de retaper les noms — et de faire une faute de frappe en partie 4.

```
# Variables réutilisées dans tout le TP, reprises du tableau de décisions ; le echo
# final est un contrôle, les trois valeurs doivent s'afficher non vides.
# Sous PowerShell, utilisez $env:RG = "..." à la place.
export SUFFIX="_______"
export LOCATION="_______"
export RG="rg-dermascan-registre-$SUFFIX"
echo "$SUFFIX | $LOCATION | $RG"
```

```
# Crée le groupe de ressources. Les étiquettes ne sont pas décoratives : elles
# servent à imputer un coût à un projet et à retrouver ce qui doit être détruit.
# Ce groupe n'est PAS celui d'une séance : il porte le registre, que les séances 2,
```

```
# 3 et 4 consomment, ainsi que le projet fil rouge. Sa date `a_detruire` est donc
# POSTÉRIEURE À LA SOUTENANCE, pas celle d'aujourd'hui. C'est la seule ressource Azure du module qui vous
# survive d'une séance à l'autre, et la seule que vous ne détruirez pas ce soir.
az group create --name "$RG" --location "$LOCATION" \
  --tags projet=dermascan role=registre proprietaire="_______" a_detruire="_______"
# Vérifie l'existence du groupe et relève son état d'approvisionnement.
az group show --name "$RG" --output table
```

Mesure 3 — état d'approvisionnement du groupe de ressources. À relever : la valeur du champ `ProvisioningState`. Que vaut-elle ?

Question 2.2 — La création du groupe de ressources a-t-elle consommé du crédit ? Regardez de nouveau l'analyse des coûts et justifiez votre réponse en une ligne.

2.3 — Vérification de Docker

```
# Client et démon : une erreur de connexion signifie que Docker Desktop est arrêté.
docker version
# État détaillé du démon : conteneurs, images, architecture, mémoire allouée.
docker info
# Test de bout en bout : téléchargement depuis Docker Hub puis exécution.
docker run --rm hello-world
```

Mesure 4 — environnement Docker du poste. Complétez ce relevé. Ces valeurs caractérisent votre environnement d'exécution et seront reportées dans la fiche d'identité en partie 5.

Élément relevé	Commande source	Votre valeur
Version du client Docker	`docker version`	
Version du démon (server)	`docker version`	
Architecture (`amd64` / `arm64`)	`docker info`	
Nombre d'images déjà présentes	`docker info`	
Mémoire allouée au démon	`docker info`	

Point d'attention architecture. Si vous relevez `arm64` (Mac Apple Silicon), notez-le en rouge : une image construite en `arm64` ne démarrera pas sur un hôte `amd64`. Ce point sera traité en partie 4.

Partie 3 — Conteneuriser l'application

3.1 — L'application fournie

L'application vous est fournie : un service Flask d'une quarantaine de lignes, accompagné d'un `requirements.txt` d'une seule ligne (`Flask==3.0.3`). Placez les deux fichiers **tels quels** dans votre dossier de travail. Vous ne les écrivez pas et vous ne les modifiez pas : l'objet du TP est de les emballer.

Ce que fait le service, et sur quoi porteront vos mesures :

Élément	Rôle	Contrainte
`GET /health`	Renvoie un JSON de santé : `status` , `service` , `version` , `hostname` , `uptime_seconds`	Code 200 tant que le processus est vivant
`GET /`	Renvoie un message d'accueil et la liste des endpoints	Aucune
`SERVICE_NAME` , `APP_VERSION` , `PORT`	Lues dans l'environnement, avec une valeur par défaut	Surchargeables au lancement, sans reconstruction
Écoute réseau	`host="0.0.0.0"` , port pris dans `PORT` (défaut 8000)	Une écoute sur `127.0.0.1` rendrait le conteneur injoignable

Extrait : la forme de la réponse de santé, celle que vous relèverez en 3.4.

```
@app.get("/health")
def health():
    return jsonify({"status": "ok", "service": SERVICE_NAME,
                    "version": APP_VERSION, "hostname": socket.gethostname(),
                    "uptime_seconds": round(time.time() - STARTED_AT, 1)})
```

Question 3.1 — L'application écoute sur `0.0.0.0` et non sur `127.0.0.1` . Pourquoi est-ce indispensable dans un conteneur ? Une phrase dans `MESURES.md` .

3.2 — Écriture du Dockerfile

Créez un fichier nommé `Dockerfile` , sans extension, à côté d' `app.py` . Vous l'écrivez vous-même : l'énoncé donne la logique et les éléments attendus, pas le fichier.

La logique, et pourquoi cet ordre. Une image se construit en couches empilées, une par instruction. Docker réutilise une couche tant que ni l'instruction ni les fichiers qu'elle copie n'ont changé ; dès qu'une couche est invalidée, **toutes celles qui la suivent sont rejouées**. L'ordre se déduit donc de la fréquence de changement : d'abord ce qui bouge rarement (image de base, réglage d'environnement, liste des dépendances et leur installation), ensuite seulement ce qui bouge à chaque modification (le code applicatif). Le durcissement — création d'un utilisateur non privilégié, puis bascule vers lui — vient après l'installation, qui a besoin des droits, et avant la déclaration du processus final.

Les éléments attendus. Dix instructions, dans un ordre que vous devez pouvoir justifier :

Instruction	Rôle	Contrainte ou décision
`FROM`	Image de base Python	Variante légère, version **explicitement fixée** ; `latest` est exclu, il rend la construction non reproductible
`WORKDIR`	Répertoire de travail du conteneur	Chemin absolu, créé s'il n'existe pas ; les instructions suivantes lui sont relatives
`ENV`	`PYTHONDONTWRITEBYTECODE=1` et `PYTHONUNBUFFERED=1`	Pas de `.pyc` dans une image jetable ; logs non tamponnés, sinon `docker logs` reste muet
`COPY` (1)	Le fichier qui change **le moins** souvent	Décision centrale pour le cache : lisez 3.5 avant de trancher
`RUN pip install`	Installation des dépendances	`--no-cache-dir`, pour ne pas embarquer le cache de pip dans la couche
`COPY` (2)	Le code applicatif	Change à chaque modification, donc copié le plus tard possible
`RUN useradd`	Création d'un utilisateur non privilégié, uid à décider au-delà de 10000, et appropriation du répertoire de travail	Un conteneur qui tourne en root est le premier point relevé en audit
`USER`	Bascule vers cet utilisateur	Le processus final ne doit pas s'exécuter en root
`EXPOSE`	Documente le port d'écoute de l'application	Purement déclaratif : seul `-p` au lancement publie un port
`CMD`	Processus principal du conteneur	Forme « exec » (tableau JSON) : en forme « shell », le processus ne reçoit pas les signaux d'arrêt

Extrait : le seul endroit où l'ordre est délicat, les deux `COPY` séparés par l'installation.

```
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py ./
```

Question 3.2 — Vous avez écrit `EXPOSE`. Si vous l'aviez oublié, le conteneur aurait-il quand même fonctionné avec `-p` ? Répondez, puis vérifiez expérimentalement si vous avez le temps.

Ajoutez également un fichier `.dockerignore` excluant au minimum `__pycache__/`, `*.pyc`, `.git/`, `.venv/` et `README.md`.

Question 3.3 — Que se passe-t-il si vous oubliez le `.dockerignore` et que votre dossier contient un `.venv/` de 400 Mo ? Quelles sont les deux conséquences — l'une sur la durée du build, l'autre sur l'image ?

3.3 — Construction et première mesure

```
# -t donne nom:tag, le "." final désigne le contexte envoyé au démon Docker.
time docker build -t "_______:_______" .
# Taille de l'image produite.
```

```
docker images | grep dermascan
# Une ligne par instruction du Dockerfile, avec la taille ajoutée par chacune.
docker history "dermascan-health:0.1.0"
```

Mesure 5 — coût du premier build et composition de l'image. Relevez vos mesures dans `MESURES.md` :

Mesure	Votre valeur
Durée du premier build (cache vide)	
Taille de l'image	
Nombre de couches listées par `docker history`	
Couche la plus lourde, et son instruction	
Taille de l'image de base seule (`docker images python`)	

Question 3.4 — Quel pourcentage de la taille finale est imputable à l'image de base seule ? Calculez-le.

3.4 — Exécution locale et test

```
# -d détache, --name nomme, -p publie <port_hote>:<port_conteneur> (port hôte libre).
docker run -d --name dermascan-health -p _______:_______ "dermascan-health:0.1.0"
# L'état doit être "Up" et non "Exited", et les logs montrer le démarrage de Flask.
docker ps
docker logs dermascan-health
# Interroge la sonde : -s masque la progression, -i affiche les en-têtes.
curl -s -i http://localhost:_______/health
```

Mesure 6 — réponse de la sonde `/health` du conteneur local. Relevez :

Mesure	Votre valeur
Code HTTP retourné	
Champ `hostname` du JSON	
Champ `version` du JSON	
Champ `status` du JSON	

Question 3.5 — D'où vient la valeur du champ `hostname` ? Relancez le conteneur (`docker rm -f` puis `docker run`) et observez si elle change. Qu'en déduisez-vous sur l'identité d'un conteneur ?

Question 3.6 — Sans reconstruire l'image, lancez un second conteneur sur un autre port hôte, en surchargeant la variable `APP_VERSION` .

```
# -e injecte une variable au lancement. Port hôte libre et différent, autre version.
docker run -d --name dermascan-health-bis -p _______:8000 \
  -e APP_VERSION="_______" "dermascan-health:0.1.0"
```

La réponse JSON change-t-elle sans qu'aucune reconstruction n'ait eu lieu ? Que cela vous dit-il sur la séparation entre l'image (l'artefact, immuable) et sa configuration (variable selon l'environnement) ?

3.5 — Ordre des instructions et cache de build

C'est le point le plus important de la partie 3. Vous allez **mesurer** ce que coûte un mauvais ordre d'instructions.

Expérience A — bon ordre (celui de votre Dockerfile actuel).

```
# Modifiez une seule ligne d'app.py : changez le texte du endpoint racine
# ("DermaScan health probe" -> "DermaScan health probe v2"), puis reconstruisez.
time docker build -t "dermascan-health:0.1.0" .
```

Observez la sortie : combien d'étapes affichent `CACHED` ? Notez-le.

Expérience B — mauvais ordre.

Créez une copie nommée `Dockerfile.naif` dans laquelle vous **fusionnez les deux COPY en un seul** placé **avant** le `RUN pip install` :

```
# Extrait de Dockerfile.naif : tout est copié d'un coup, avant l'installation.
COPY . ./
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Construit avec le Dockerfile naïf sous un tag distinct, pour ne pas écraser le bon.
docker build -f Dockerfile.naif -t "dermascan-health:naif" .
# Modifiez de nouveau une ligne d'app.py, puis reconstruisez et chronométrez.
time docker build -f Dockerfile.naif -t "dermascan-health:naif" .
```

Mesure 7 — effet de l'ordre des instructions sur le cache de build. Complétez le tableau comparatif :

Scénario	Durée du build	Nombre d'étapes `CACHED`	`pip install` rejoué ?
A — bon ordre, après modification d' `app.py`			
B — mauvais ordre, après modification d' `app.py`			
Écart A/B			

Question 3.7 — Expliquez en trois à cinq lignes le mécanisme d'invalidation du cache de Docker : quand une couche est-elle réutilisée, et que se passe-t-il pour toutes les couches suivantes dès qu'une couche est invalidée ?

Question 3.8 — En séance 2, votre `requirements.txt` contiendra `scikit-learn`, `pandas` et `numpy`. Estimez, à partir de votre mesure d'écart, ce que le mauvais ordre coûterait sur vingt itérations de développement dans une journée.

3.6 — Contrôle de l'utilisateur non-root

```
# Identité sous laquelle tourne le processus dans le conteneur.
docker exec dermascan-health id
# Utilisateur configuré dans l'image elle-même, sans conteneur en cours.
docker image inspect "dermascan-health:0.1.0" --format '{{.Config.User}}'
# Test négatif : un conteneur correctement durci doit refuser cette écriture.
docker exec dermascan-health touch /etc/preuve_root
```

Mesure 8 — contrôles de l'utilisateur non privilégié dans le conteneur.

Contrôle	Résultat attendu	Votre observation
`id` dans le conteneur	uid non nul	
`.Config.User` de l'image	non vide	
Écriture dans `/etc`	refusée	

Question 3.9 — Un attaquant qui obtient l'exécution de code arbitraire dans votre conteneur : que gagne-t-il de plus si le processus tourne en root plutôt qu'en `appuser` ? Deux lignes.

Partie 4 — Azure Container Registry

Votre image n'existe que sur votre poste. Elle est donc inutilisable par la séance 3, par un collègue, et par Azure. Il lui faut un registre.

4.1 — Création du registre

```
# Nom décidé au tableau de décisions : minuscules et chiffres, unique sur Azure.
export ACR_NAME="_______"
# Disponibilité du nom AVANT création : lisez "nameAvailable" puis "message".
az acr check-name --name "$ACR_NAME" --output table
# Le SKU fixe le quota de stockage, les fonctions disponibles et le coût mensuel.
az acr create --resource-group "$RG" --name "$ACR_NAME" --sku "_______" --location
"$LOCATION"
# Caractéristiques du registre créé, dont son serveur de connexion.
az acr show --name "$ACR_NAME" --output table
```

Mesure 9 — caractéristiques du registre ACR créé. À relever :

Élément	Votre valeur
`loginServer` du registre	
SKU retenu	
État d'approvisionnement	
Coût mensuel annoncé pour ce SKU (documentation tarifaire Azure)	

Question 4.1 — Comparez les trois SKU (Basic, Standard, Premium) sur deux critères : quota de stockage inclus et fonctionnalités. À partir de quel besoin concret faudrait-il passer au Premium pour DermaScan ?

4.2 — Authentification sur le registre

```
# Authentifie Docker auprès d'ACR via le jeton "az login" : aucun mot de passe saisi.
az acr login --name "$ACR_NAME"
```

Question 4.2 — Ouvrez le fichier `~/.docker/config.json`. Qu'y trouve-t-on après un `az acr login` réussi ? Pourquoi ce fichier est-il un secret à ne jamais committer ?

Question 4.3 — Il existe une alternative : activer l'utilisateur administrateur du registre (`az acr update --admin-enabled true`) et se connecter avec un mot de passe. Citez deux raisons pour lesquelles cette méthode est déconseillée en entreprise, alors qu'elle est plus simple.

4.3 — Étiquetage et publication

Une image poussée sur un registre doit porter un nom **complet** : `<loginServer>/<dépôt>:<tag>`. Sans ce préfixe, Docker tenterait de pousser sur Docker Hub.

```
# Nom complet <loginServer>/<dépôt>:<tag>, construit depuis le registre.
export LOGIN_SERVER=$(az acr show --name "$ACR_NAME" --query loginServer --output tsv)
export IMAGE_FULL="$LOGIN_SERVER/_______:_______"
echo "$IMAGE_FULL"
# docker tag ne copie et ne reconstruit rien : même image, second nom, un seul IMAGE ID.
docker tag "dermascan-health:0.1.0" "$IMAGE_FULL"
docker images | grep dermascan
# Envoi vers le registre, couche par couche.
time docker push "$IMAGE_FULL"
```

Mesure 10 — coût du premier push vers ACR et identité de l'image poussée.

Mesure	Votre valeur
Durée du push	
Nombre de couches poussées	
`IMAGE ID` local et `IMAGE ID` taggé : identiques ?	
Digest (`sha256:...`) retourné en fin de push	

Expérience complémentaire. Poussez une seconde fois la même image, sans rien changer : `time docker push "$IMAGE_FULL"`.

Question 4.4 — La durée est-elle la même ? Qu'affiche Docker pour chaque couche, et quel mécanisme cela révèle-t-il sur le stockage des couches dans un registre ?

Point architecture. Si vous avez relevé `arm64` en partie 2.3, votre image ne tournera pas sur les hôtes Linux `amd64` d'Azure utilisés en séance 3. Reconstruisez-la explicitement pour l'architecture cible du cloud, puis poussez de nouveau :

```
# --platform fixe l'architecture cible. --provenance=false supprime les manifestes
# d'attestation que le constructeur par defaut ajoute : sans lui, le tag ne porte
# plus d'architecture lisible et le relevé de la partie 4.4 devient inexploitable.
docker build --platform linux/amd64 --provenance=false -t "$IMAGE_FULL" .
docker push "$IMAGE_FULL"
```

4.4 — Vérification dans le registre

Un registre conserve **tous** les manifestes poussés, y compris ceux qui ne portent plus de tag. Si vous avez poussé une première image `arm64` avant de la reconstruire en `amd64`, les deux coexistent : une seule ligne porte le tag, et c'est celle-là qui fait foi. Une ligne sans tag est un manifeste orphelin, pas une erreur.

```
# Dépôts présents dans le registre, puis tags d'un dépôt donné.
az acr repository list --name "$ACR_NAME" --output table
az acr repository show-tags --name "$ACR_NAME" --repository "_______" --output table
# Manifeste : digest, date de création, taille, architecture et OS cibles.
az acr manifest list-metadata --registry "$ACR_NAME" --name "dermascan-health" --output
table
```

Vérifiez ensuite **dans le portail Azure** : votre registre, puis `Services` → `Référentiels`. Faites une capture d'écran montrant le dépôt et son tag. Elle fait partie du livrable.

Test final de bout en bout. Prouvez que l'image du registre est exécutable, et pas seulement stockée :

```
# Supprime l'image locale pour forcer un téléchargement depuis ACR.
docker rm -f dermascan-health
docker rmi -f "$IMAGE_FULL" "dermascan-health:0.1.0"
# Retéléchargement et exécution. Choisissez de nouveau un port hôte libre.
docker run -d --name dermascan-from-acr -p _______:8000 "$IMAGE_FULL"
curl -s http://localhost:_______/health
```

Mesure 11 — exécution de l'image retéléchargée depuis ACR.

Mesure	Votre valeur
Durée du `docker run` avec téléchargement	
Le JSON de santé est-il identique à celui de la partie 3.4 ?	
Le `hostname` a-t-il changé ?	

Partie 5 — Gouvernance : fiche d'identité du conteneur

La gouvernance est un **fil transversal** du module : elle revient à chaque séance, par tranches courtes, plutôt qu'en un bloc indigeste en fin de semestre. Aujourd'hui, vous produisez la première brique : la fiche d'identité d'un artefact.

Pourquoi maintenant. Dans six mois, un auditeur vous demandera quelle version d'une bibliothèque tournait en production le jour d'une décision contestée. Si l'information n'a pas été écrite au moment de la construction, elle est perdue. C'est exactement le même raisonnement qui produira le **registre de modèles** (model registry) que vous mettrez en place plus tard dans le module : une fiche d'identité de conteneur est un registre de modèles dont on a retiré le modèle.

5.1 — Collecte des informations

Les commandes suivantes fournissent la matière. Ne recopiez pas votre mémoire : interrogez l'artefact.

```
# Métadonnées complètes, puis extraction ciblée des rubriques 8 et 9 de la fiche.
docker image inspect "$IMAGE_FULL"
docker image inspect "$IMAGE_FULL" \
  --format 'cree={{.Created}} arch={{.Architecture}} os={{.Os}} user={{.Config.User}}'
# Digest immuable : contrairement à un tag, il identifie un contenu définitivement.
docker image inspect "$IMAGE_FULL" --format '{{index .RepoDigests 0}}'
# Version de Python, puis inventaire réel des paquets : c'est lui qui fait foi
# pour un audit, pas le requirements.txt.
docker run --rm "$IMAGE_FULL" python --version
docker run --rm "$IMAGE_FULL" pip freeze
```

Question 5.1 — Comparez le `pip freeze` au contenu de votre `requirements.txt`. Combien de paquets supplémentaires apparaissent ? D'où viennent-ils, et pourquoi un audit de licences doit-il porter sur le `pip freeze` et non sur le `requirements.txt` ?

Question 5.2 — Identifiez trois licences : celle de Flask, celle de CPython, et celle des paquets système de la distribution Linux contenue dans l'image de base. Laquelle des trois entraîne les obligations les plus contraignantes si DermaScan redistribue cette image à un centre de dépistage client ? Justifiez en deux lignes.

5.2 — Rédaction de la fiche

Créez `FICHE_IDENTITE.md`. Elle doit contenir **exactement** les rubriques suivantes, chacune renseignée avec une valeur mesurée, jamais supposée :

#	Rubrique	Source
1	Nom de l'artefact et tag	votre tableau de décisions
2	Digest `sha256`	`docker image inspect`
3	Image de base, avec son tag exact	votre `Dockerfile`
4	Version de Python embarquée	`python --version` dans le conteneur

5	Dépendances directes et leurs versions	`requirements.txt`
6	Dépendances transitives	`pip freeze`
7	Licences identifiées et obligations associées	recherche documentaire
8	Architecture et OS cibles	`docker image inspect`
9	Utilisateur d'exécution (uid)	`docker image inspect`
10	Port exposé et endpoint de santé	votre `Dockerfile` et `app.py`
11	Producteur : nom, promotion, date et heure de construction	vous
12	Registre de publication : `loginServer` et dépôt	`az acr show`
13	Destination connue : séance 2, base de l'API d'inférence	l'énoncé
14	Limites connues et risques résiduels	votre analyse

La rubrique 14 est celle qui distingue une fiche honnête d'une fiche de façade. Vous devez y inscrire au minimum **trois** limites réelles de ce que vous avez produit. Pour vous lancer, deux pistes : l'image a-t-elle été scannée à la recherche de vulnérabilités ? le serveur de développement de Flask est-il un serveur de production ?

Format. Un tableau markdown à deux colonnes, rubrique et valeur. Aucune rubrique vide, aucune valeur « N/A » non justifiée.

Nettoyage obligatoire

Un crédit étudiant de 100 $ se consomme silencieusement. Un registre Basic oublié pendant six mois consomme une partie non négligeable de votre budget de module, et vous n'aurez aucune alerte.

Le nettoyage n'est pas la destruction aveugle de ce que vous avez produit : c'est la **décision explicite** de ce qui survit et de ce qui disparaît, avec sa raison et sa date. Ce soir, le local disparaît entièrement et le registre survit — parce que les séances 2, 3 et 4 le consomment, et le projet fil rouge après elles. C'est cette décision, et sa trace, qui sont évaluées.

Pourquoi cette section décide de la suite du module. Votre abonnement *Azure for Students* n'est pas seulement une réserve d'argent : c'est lui qui vous donne accès à `Microsoft.ContainerRegistry` et `Microsoft.ContainerInstance`. **Un crédit épuisé ne vous laisse pas un abonnement pauvre : il le fait basculer sur l'offre *Starter*, qui interdit purement et simplement ces deux fournisseurs.** Vous n'y perdez pas la possibilité de payer, vous y perdez votre registre — donc l'artefact que les séances suivantes consomment. Épuiser son crédit en séance 3, c'est ne plus pouvoir faire la séance 4, la séance 5, ni le projet fil rouge. Le nettoyage n'est pas une bonne pratique de plus : c'est ce qui vous permet de terminer le module.

Ne lancez cette section qu'après avoir terminé vos captures d'écran et votre fiche d'identité.

```
# Conteneurs locaux créés pendant la séance.
docker rm -f dermascan-health dermascan-health-bis dermascan-from-acr 2>/dev/null
# Images locales du TP.
docker rmi -f "dermascan-health:naif" "$IMAGE_FULL" 2>/dev/null
# Couches et caches orphelins : relevez l'espace libéré annoncé.
docker system prune -f
```

Côté Azure, vous ne détruisez rien ce soir — et c'est une décision, pas un oubli. Votre registre est le seul artefact que les séances 2 et 3 consomment : le détruire maintenant vous obligerait à tout reconstruire au début de la séance 2. Ce qui est évalué ici n'est donc pas le geste de destruction, c'est la **décision tracée** : savoir ce qui reste actif, ce que cela coûte par jour, et à quelle date cela doit disparaître.

```
# Inventaire de ce que vous laissez actif : on ne laisse jamais tourner ce qu'on
# n'a pas lu. Une seule ressource doit apparaître, le registre.
az resource list --resource-group "$RG" --output table
# Les étiquettes portent la décision : qui, pourquoi, jusqu'à quand.
az group show --name "$RG" --query "tags" --output json
# Contrôle global : AUCUN autre groupe que celui-ci ne doit vous appartenir.
az group list --output table
```

La commande de destruction, vous ne la lancez pas aujourd'hui, mais vous devez la connaître — c'est elle qui clôturera la séance 3, sur un groupe qui coûtera bien plus cher que celui-ci, puis ce groupe-ci à la toute fin du module :

```
# NE LA LANCEZ PAS AUJOURD'HUI. Elle supprime le groupe et TOUT son contenu,
# de façon irréversible, et votre registre avec.
# az group delete --name "$RG" --yes --no-wait
```

```
# `--no-wait` rend la main AVANT que la suppression n'aboutisse : `az group list`
# peut encore afficher le groupe pendant plusieurs minutes. Seul `az group exists`
# tranche, et il doit finir par renvoyer false.
# az group exists --name "$RG"
```

Mesure 12 — nettoyage local, et coût de ce que vous laissez actif.

Contrôle de fin de séance	Votre observation
Espace disque libéré par `docker system prune`	
Nombre de ressources laissées actives dans `$RG`	
SKU du registre, et son coût journalier estimé	
Date portée par l'étiquette `a_detruire`	
Nombre de groupes de ressources vous appartenant au total	
Crédit restant, relevé dans `Cost Management`	

La dernière ligne ouvre une série : vous la relèverez à chaque fin de séance jusqu'à la soutenance, et c'est l'**écart** entre deux relevés qui vous dira si quelque chose tourne à votre insu.

Question 6.1 — Vous laissez un registre actif pendant plusieurs semaines. Écrivez la phrase exacte que vous mettriez dans un ticket d'équipe pour que quelqu'un d'autre que vous puisse décider de le détruire sans vous appeler. Elle doit contenir trois informations : quoi, pourquoi, et jusqu'à quand.

Question 6.2 — La commande de destruction porte `--no-wait`, qui rend la main immédiatement. Quel serait le risque de considérer la suppression comme acquise à cet instant précis, et comment vous en assureriez-vous réellement ? Vous appliquerez votre réponse en fin de séance 3.

Avant de quitter la salle — extinction

Toute ressource active est une ressource que vous payez. Vous payez la *réservation*, pas l'usage : un service que personne n'appelle, la nuit et le week-end, coûte exactement le même prix qu'un service sollicité. C'est cette règle, et elle seule, qui décide si vous finirez le module avec du crédit.

Trois gestes, dans cet ordre. Aucun n'est facultatif, et ils se font **avant** de fermer votre session, pas « plus tard depuis chez vous » — un oubli du vendredi se paie jusqu'au lundi.

1. **Côté Azure. il ne reste que le registre**, et c'est voulu : les séances 2, 3 et 4 le consomment, puis le projet fil rouge. Vérifiez qu'il est bien seul, et que son étiquette `a_detruire` porte une date postérieure à la soutenance. Tout autre groupe que le vôtre est un résidu : détruisez-le maintenant.
2. **Côté poste — arrêtez ce qui tourne encore.** Conteneurs de la séance supprimés, Docker Desktop **quitté** (un démon laissé actif relance vos conteneurs au prochain démarrage et consomme mémoire et batterie), environnement virtuel désactivé.
3. **Éteignez votre machine.** Éteinte, pas en veille.

```
docker ps -a                # plus aucun conteneur de la séance
az group list --output table # un seul groupe, celui du registre
```

Contrôle d'extinction	Fait
`az group list` ne montre qu'un groupe, celui du registre	
Aucun conteneur de la séance dans `docker ps -a`	
Docker Desktop quitté	
Machine éteinte	

Livrable attendu

Un dossier `s1-<votre-nom>/` déposé sur le dépôt de classe avant la fin de la séance, contenant :

Fichier	Contenu
`reponses_partie1.md`	Les quatre tableaux de la partie 1 complétés, la mesure 1 et les questions 0.1 à 1.3
`Dockerfile`	Votre Dockerfile final, les dix instructions attendues dans l'ordre que vous justifiez
`Dockerfile.naif`	La variante de l'expérience B
`.dockerignore`	Le fichier d'exclusion
`app.py`, `requirements.txt`	Les fichiers fournis, inchangés
`MESURES.md`	Les mesures 2 à 12 (parties 2, 3, 4 et nettoyage), plus vos réponses aux questions 2.1 à 6.2
`FICHE_IDENTITE.md`	Les 14 rubriques de la partie 5
`captures/`	Au minimum : le dépôt visible dans le portail ACR, et la sortie de `curl /health`

Ce qui n'est pas demandé et ne sera pas noté : un modèle de machine learning, une API d'inférence, un déploiement de service managé. Ces trois éléments sont l'objet des séances 2 et 3. Les produire aujourd'hui ne rapporte aucun point.

Annexe A — Dépannage

Symptôme	Cause la plus fréquente	Action
`az: command not found`	CLI absent du PATH	Réinstaller, rouvrir le terminal
`az login` ouvre le navigateur puis boucle	Compte scolaire avec MFA	`az login --use-device-code`
`Cannot connect to the Docker daemon`	Docker Desktop non démarré	Lancer Docker Desktop, attendre l'icône verte
`denied: requested access to the resource is denied` au push	Image non taggée avec le `loginServer`	Vérifier `echo "$IMAGE_FULL"`
`unauthorized: authentication required`	Jeton ACR expiré (3 h)	Rejouer `az acr login --name "$ACR_NAME"`
`The registry name is already in use`	Nom ACR non unique sur Azure	Changer de suffixe, relancer `az acr check-name`
Port déjà utilisé au `docker run`	Port hôte occupé	Changer le port hôte de `-p`, le port conteneur reste inchangé
`exec format error` au démarrage sur Azure	Image construite en `arm64`	Reconstruire avec `--platform linux/amd64`
`curl` ne répond pas mais `docker ps` affiche `Up`	Application liée à `127.0.0.1`	Vérifier `host="0.0.0.0"` dans `app.py`

Annexe B — Plan de repli sans Azure

Si votre abonnement refuse la création du registre, la compétence reste acquise et notée. Lancez un registre local :

```
# Registre conforme à l'API Docker Registry v2, en local sur le port 5000.
docker run -d -p 5000:5000 --name registry-local registry:2
# Le nom complet devient localhost:5000/<dépôt>:<tag> ; vérification via l'API v2.
docker tag "dermascan-health:0.1.0" "localhost:5000/dermascan-health:0.1.0"
docker push "localhost:5000/dermascan-health:0.1.0"
curl -s http://localhost:5000/v2/_catalog
```

Mentionnez explicitement ce repli dans `MESURES.md` et dans la rubrique 12 de votre fiche d'identité. Les parties 4.1 et 4.2 seront évaluées sur vos réponses écrites aux questions 4.1 à 4.3 plutôt que sur l'artefact.

Annexe C — Ce que vous emportez vers la séance 2

Acquis de S1	Usage en S2
Le `Dockerfile` avec son ordre d'instructions	Réutilisé tel quel, `requirements.txt` enrichi de scikit-learn
Le registre ACR, **conservé** dans `rg-dermascan-registre-<suffixe>`, et le réflexe `az acr login`	L'API d'inférence y sera publiée. Ne détruisez pas ce groupe avant la fin de la séance 3
La fiche d'identité du conteneur	Devient une *model card* : mêmes rubriques, plus le modèle, ses données d'entraînement et ses métriques
Le réflexe de nettoyage	Appliqué à chaque fin de séance jusqu'à la soutenance